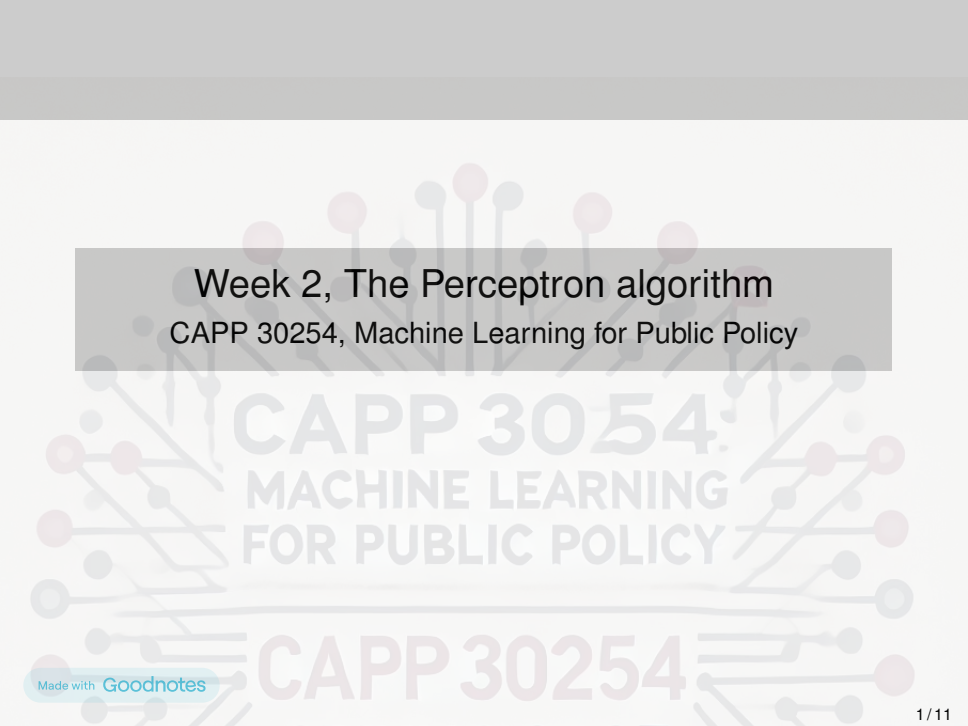




Week 2, The Perceptron algorithm

CAPP 30254, Machine Learning for Public Policy

Reminder: Linear classifiers

Recall a linear binary classifier makes predictions:

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w}^T \mathbf{x} \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

given a weight vector $\mathbf{w}$.

/
different models
find different $\mathbf{w}$

$$\underline{\mathbf{w}^T \mathbf{x}} = w_1 x_1 + \dots + w_p x_p \in \mathbb{R}$$

$\geq +1$ cat
 < -1 dog

Linearly separable data

2 classes

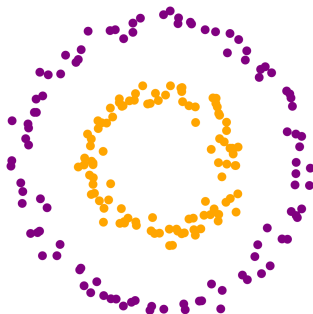
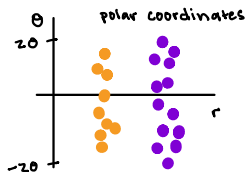
A dataset is called *linearly separable* when we can draw a line in 2D that perfectly separates the data into different classes. The separator is a plane in 3D or a hyperplane in higher dimensions.



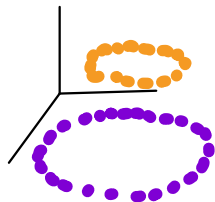
¹Generated by GPT-4, DALL-E 3 after the prompt: "can you draw a linearly separable dataset in 2D?"

What if data is not linearly separable?

Some data that doesn't look linearly separable can be transformed to be separable. For example, consider this circular data with different radii where two classes have different radii.



Later: kernel methods



2

²Generated by GPT-4, DALL-E 3 after the prompt: "Can you draw circular non-linearly separable data set?"

The Perceptron algorithm

supervised binary classification problem

“invention”

old! 1958

The input to the Perceptron algorithm is a set of training samples $(\mathbf{x}_i, y_i)$ for $i = 1, \dots, n$ where each $\mathbf{x}_i \in \mathbb{R}^p$ is a feature vector and $y_i \in \{+1, -1\}$ is a class label.

Perceptron is an algorithm that iteratively learns a weight vector $\mathbf{w}$ that separates the two classes of a binary classification problem.

If the training data is linearly separable, Perceptron will find a weight vector $\mathbf{w}$ that perfectly separates the classes.



³Generated by GPT-4, DALL-E 3 after the prompt: “could you give another example of linearly separable data with purple and orange circle”

The bias term

$$\underline{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_p \end{bmatrix} \quad \underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} \quad x_i \text{ is the value of feature } i \quad \underline{w}^T \underline{x} \approx y = mx + \underbrace{b}_{\text{bias}}$$

The *bias term* is a constant value that is added to the weight vector $\mathbf{w}$. Without it, the model is forced to pass through the origin which is unreasonable. With it, the model is able to shift in space, better fitting the data.

The bias term can be confusing notationally. The bias can be written as an extra term, as in $\mathbf{w}^T \mathbf{x} + b \approx mx + b$

$$\underline{w}, \underline{x} \in \mathbb{R}^p$$

Other times it is implicitly part of a training sample and weight vector, as in $\mathbf{w}^T \mathbf{x}$.

$$\underline{w}, \underline{x} \in \mathbb{R}^{p+1}$$

$$\underline{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_p \\ w_{p+1} \end{bmatrix}$$

$$\underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \\ 1 \end{bmatrix}$$

$$\text{or } \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{bmatrix}$$

$\underline{x}^T \underline{w}$ ← easier notation

$$X = [x | 1] = \begin{bmatrix} -x_1^T & -1 \\ \vdots & \vdots \\ -x_n^T & -1 \end{bmatrix}$$

The Perceptron algorithm

Initialize weight vector $\mathbf{w}$ and bias term b to zero.

For each training example $(\mathbf{x}_i, y_i)$:

- Compute the prediction: $\hat{y}_i = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$

$$\begin{aligned} \mathbf{w}^T \mathbf{x} \geq 0 &\rightarrow +1 \\ < 0 &\rightarrow -1 \end{aligned}$$

- If the prediction is wrong ($y_i \neq \hat{y}_i$):

 - Update the weights $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$

correct: do nothing

wrong: change $\mathbf{w}$

"pull" $\mathbf{w}$ toward

correct examples

 - Update the bias term $b \leftarrow b + \eta y_i$

Repeat over the dataset for multiple epochs until convergence or until a maximum number of iterations is reached.

one pass over data set

no mistakes

Just as in gradient descent, η is a learning rate. Sometimes

most?

$\eta = 1$.

Perceptron's relation to ??? stochastic gradient descent

Initialize weight vector $\mathbf{w}$ and bias term b to zero.

For each training example $(\mathbf{x}_i, y_i)$:

- Compute the prediction: $\hat{y}_i = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$

- If the prediction is wrong ($y_i \neq \hat{y}_i$):

 - Update the weights $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$

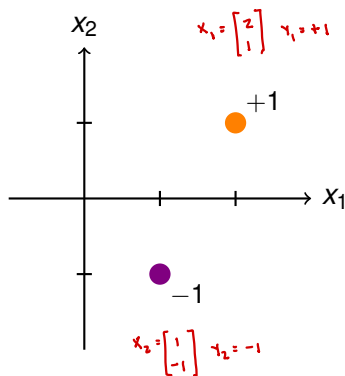
 - Update the bias term $b \leftarrow b + \eta y_i$

} $\nabla_{\mathbf{w}} \ell(\mathbf{w})$

Repeat over the dataset for multiple epochs until convergence or until a maximum number of iterations is reached.

$$\nabla_{\mathbf{w}} \ell(\mathbf{w}) = \max(0, -y_i(\mathbf{w}^T \mathbf{x}_i))$$

Example



start: $\underline{w} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ $b = 0$ $\eta = 1$

epoch 1: correct

$$\hat{y}_1 = 0 \cdot 2 + 0 \cdot 1 + 0 = 0 \geq 0 \rightarrow 1$$

$$\hat{y}_2 = 0 \cdot 1 + 0 \cdot (-1) + 0 = 0 \rightarrow -1$$

incorrect

update:

$$\underline{w} = \underline{w} - \eta_i \underline{x}_i$$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix} + (-1) \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

$$b = b + \eta_i$$

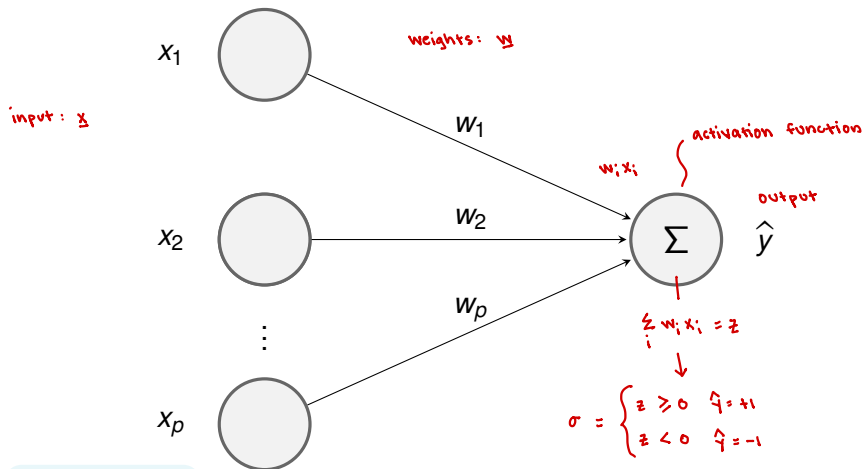
$$= 0 + -1 = -1$$

epoch 2:

TRY

Linear models as neural networks

A linear model can be viewed as a very simple neural network.





4

⁴Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalist logo for a machine learning class...”